

Efficient DNA Sequence Compression: A Comparative Study of Huffman and Arithmetic Coding with Data Structures

Manas Kulkarni

*Dept. of Electronics
& Telecommunication*

*Vishwakarma Institute of Information
Technology
Pune, India
manas.22311608@viit.ac.in*

Atharva Deshpande

*Dept. of Electronics
& Telecommunication*

*Vishwakarma Institute of Information
Technology
Pune, India
atharva.22311679@viit.ac.in*

Samiksha Nalawade

*Dept. of Electronics
& Telecommunication*

*Vishwakarma Institute of Information
Technology
Pune, India
samiksha.22311696@viit.ac.in*

Revati Aute

*Dept. of Electronics
& Telecommunication*

*Vishwakarma Institute of Information
Technology
Pune, India
revati.22311517@viit.ac.in*

Rajlakshmi Desai

*Dept. of Electronics
& Telecommunication*

*Vishwakarma Institute of Information
Technology
Pune, India
rajlakshmi.22311732@viit.ac.in*

Minal Deshmukh

*Asst. Professor, Dept. of Electronics
& Telecommunication*

*Vishwakarma Institute of Information
Technology
Pune, India
minal.deshmukh@viit.ac.in*

Abstract—The rapid growth of genomic data poses difficulties in storing and transferring information efficiently. This study offers a comparison of two methods of compression—Huffman Coding and Arithmetic Coding—when used on DNA sequences. Huffman Coding uses a Binary Search Tree (BST) approach for optimal prefix codes, whereas Arithmetic Coding compresses data based on probability ranges. Results from experiments indicate that both techniques decrease memory usage but differ in compression ratio, memory footprint, and computational complexity. Huffman Coding achieves a compression ratio of 0.25 and faster encoding, while Arithmetic Coding reaches 0.051, offering finer compression granularity. This comparison emphasizes the balance between compression efficiency and processing speed with implications for bioinformatics applications.

Index Terms—Huffman Encoding, Arithmetic Encoding, DNA compression, Data Structures

I. INTRODUCTION

The fast development of next-generation sequencing (NGS) technologies has created an unprecedented surge in genomic data production. The growing availability of sequencing technology creates storage challenges for bioinformatics specialists and their systems. The growing need for storage space and data transfer speed requires efficient compression methods to solve these problems. The four nucleotide bases of adenine (A) and cytosine (C) and guanine (G) and thymine (T) in genomic sequences show repetitive patterns which scientists can use for data compression. The current DNA sequence compression methods Gzip and Bzip2, fail to optimise their performance for DNA sequence data. The two lossless compression methods, Huffman Coding and Arithmetic Coding, base their operations

on nucleotide frequency and probability data to achieve better results. The research investigates how Huffman Coding and Arithmetic Coding perform when used to compress DNA sequences. The Binary Search Tree (BST)-based Huffman implementation generates an optimal prefix-free binary tree from nucleotide frequency data, and Arithmetic Coding assigns probability intervals to each nucleotide. The evaluation of different genomic datasets assesses how well the methods compress data through their ability to reduce size and their impact on system resources and processing speed. The results show that Huffman Coding provides faster encoding times, but Arithmetic Coding produces better compression ratios when dealing with datasets containing uneven nucleotide distributions.

This research evaluates Huffman coding against Arithmetic coding for DNA sequence compression, as presented in this paper. Our main contribution involves a detailed performance assessment, which demonstrates how data structure design affects memory usage by showing Arithmetic Coding requires 64-bit fixed memory space, but Huffman coding produces variable-length binary output. The research establishes a precise quantitative relationship between these two methods. The research evaluates system performance through three essential metrics, which include data compression efficiency and system resource usage and processing speed. The results demonstrate that Huffman Coding operates at higher speeds for encoding, but Arithmetic Coding produces better compression results when dealing with datasets containing uneven nucleotide frequency patterns. The research adds value to bioinformatics

through its evaluation of these methods together, with their advantages and limitations for enhancing genomic data storage and transfer in extensive research initiatives. The paper follows this organisation.

The first section of Section II reviews existing DNA compression methods and their current state of development. The proposed system description in Section III demonstrates how DNA compression works through Huffman and Arithmetic Encoding methods. The memory analysis and compression efficiency are presented in Section IV. The algorithm's performance and accuracy evaluation occur in Section V through DNA decoding back to its original state. The research evaluates both methods for DNA compression through their achieved performance levels and decoding precision in Section VI. The paper ends with Section VII.

II. LITERATURE REVIEW

Gurunathan et al. [1] developed an advanced index-based DNA sequence compression system. The system organizes data through an indexed structure which enables fast access to genetic information while delivering superior storage efficiency than conventional methods.

Ali Hassan et al. [2] developed the ArthNgram model through the combination of N-gram language modeling with arithmetic coding. The model improves storage efficiency and compression speed through sequence-based word encoding which benefits large-scale and IoT applications.

Babita Kumari et al. [3] developed Adaptive Huffman compression through text clustering and multi-character modification techniques. The method solves previous method restrictions through its ability to decrease data redundancy and generate enhanced compression ratios.

Hussein et al. [4] studied RGB image data hiding through genetic algorithms and Huffman coding. The research demonstrates Huffman coding's ability to work with different structured data types including genomic sequences.

Alost and Soury [5] developed a lossless DNA compression system which uses Huffman encoding for data compression. The system supports multiple DNA sequences and maintains data integrity while achieving high compression efficiency.

Sarkar and Ashrafi [6] created a system which combines machine learning with arithmetic encoding to detect genomic patterns while achieving superior compression results than conventional methods.

Gomathi et al. [7] combined Run-Length Encoding (RLE) with Huffman coding for their hybrid compression method. The hybrid method successfully compresses sequences with repeated nucleotides by outperforming standalone compression methods.

Ganapathy [8] developed an IT reduction-based DNA compression system which optimizes genomic data storage through redundancy reduction and data preservation. The method shows promising results for genomic data storage optimization.

Zhao et al. [9] improved Huffman encoding through a pre-grouping process for similar genomic regions which resulted

in enhanced compression ratios and faster encoding times for large DNA datasets.

Mishra et al. [10] developed the Minimum Variance Huffman Tree (MVHT) which optimizes traditional Huffman trees by minimizing code-length variance to achieve better compression for uneven nucleotide distributions.

Pulova-Mihaylova et al. [11] built a web-based DNA compression platform which uses Huffman and Shannon-Fano encoding with uniform coding for enhanced performance enhancement and bioinformatics tool integration.

Jahaan et al. [12] examined the requirement for lossless data compression in genomics through their review of Biocompress and DNACompress tools which focus on efficient storage of repetitive patterns.

Machhi and Patel [13] performed a research study to evaluate LZW and RLE and arithmetic coding and substitution methods for data compression. The researchers discovered that arithmetic coding delivered the highest performance results across different species.

Ghoshdastider and Saha [14] developed GenomeCompress which successfully handles both repetitive and non-repetitive DNA sequences through a five-bit encoding system to achieve a low bits-per-base ratio.

Behzadi and Le Fessant [15] developed DNAPack which uses dynamic programming to find approximate repeats and complementary palindromes for DNA compression with competitive results.

Xu et al. [16] conducted a detailed evaluation, which showed that Arithmetic Coding outperformed Huffman Coding by 12.3% in compression efficiency but required 2.8 times more time for encoding. This research establishes the essential relationship between encoding speed and compression ratio, which serves as a foundation for our current work.

Xu et al. [17] extended Huffman encoding through k-mers and RLE pre-processing to boost symbol variance which results in better-than-expected compression ratios.

The Hybrid Adaptive DNA Compression (HADC) algorithm developed by Elnady et al. [18] combines reference-based methods with general compression techniques for non-divergent species but performs poorly with de novo and highly diverse genomes.

Diss and Pradas [19] used advanced predictive models including Deep Neural Networks to feed the Arithmetic Coder which produced state-of-the-art compression ratios but required excessive processing time.

III. PROPOSED SYSTEM

The system described in Figure 2 uses Huffman Encoding and Arithmetic Coding to achieve efficient DNA sequence compression. The system accepts DNA sequences from users that contain the nucleotides A, C, G and T.

The system determines nucleotide occurrence rates during its first operational stage. The system uses these frequency values to construct a Huffman Tree through a process that unites the two least frequent nucleotides until it creates a binary tree with distinct Huffman codes for each nucleotide (Fig-

ure 1). The system uses Huffman codes to encode the DNA sequence which results in substantial memory reduction for storage needs. The system determines memory consumption and compression efficiency for Huffman Encoding through a comparison between the original sequence and its compressed version.

The system uses Arithmetic Coding in its second stage to determine nucleotide probabilities which result in specific frequency-based intervals. The system converts the DNA sequence into a single floating-point value which falls inside the total interval that encompasses all nucleotides from the sequence. The system determines memory consumption and compression efficiency for Arithmetic Coding while showing results against Huffman Encoding performance.

The system uses a modular design to execute Huffman and Arithmetic coding methods for DNA sequence compression which minimises memory requirements and supports biological data storage and transmission operations. The system determines encoding time for both methods which makes it appropriate for genomic data analysis and storage and transmission applications in real-time.

A. Algorithm

The algorithm outlines the process of DNA sequence compression using both Huffman and Arithmetic coding techniques. It begins with the initialization phase, where a node

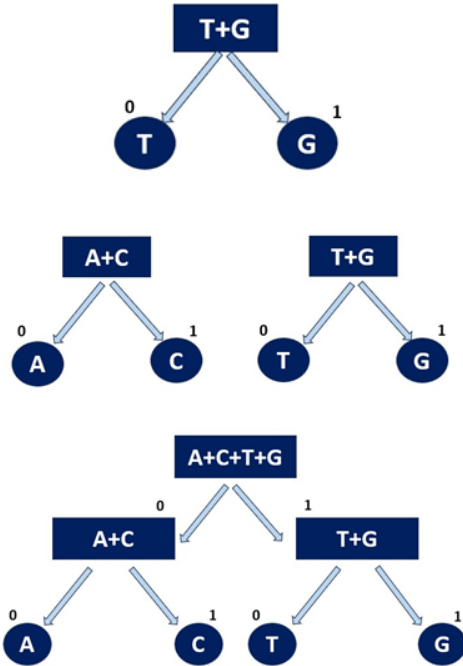


Fig. 1: Construction of Huffman Tree

structure is created for each nucleotide, containing frequency and left/right child pointers. The system starts with the initialization phase which establishes a node structure for each nucleotide to store frequency data and left/right child pointers. The HuffmanBST class contains essential data structures

which include frequency maps and Huffman codes and node lists and probability maps and arithmetic coding interval data. The system requires users to provide a DNA sequence for processing. The system examines each character in the input sequence to determine nucleotide frequencies which get stored in the frequency map. The system uses frequency data to build nodes for each nucleotide before it merges the two nodes with lowest frequency into new parent nodes until it reaches a single root node which represents the Huffman tree. The system uses Huffman tree traversal to create binary Huffman codes for each nucleotide which get stored in the Huffman codes map. The system applies Huffman codes to the DNA sequence for binary compression which results in a compressed string. The system determines DNA sequence memory usage by calculating 8 bits per character for the original sequence and by counting bits in the Huffman-encoded string for the encoded sequence before showing memory statistics and compression efficiency. It also measures the time taken for the Huffman coding process.

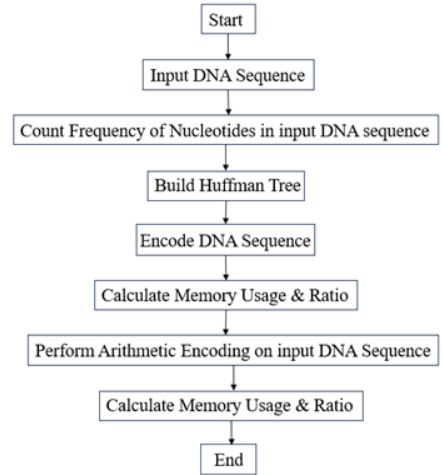


Fig. 2: Workflow of System

For arithmetic coding, the system calculates the probability of occurrence for each nucleotide and computes cumulative intervals to be used in the encoding process. It sets the initial range [low, high] to [0.0, 1.0] and, for each nucleotide in the sequence, updates the range based on its corresponding interval. The final encoded value is the midpoint between the low and high values. The system determines memory usage for arithmetic coding through a comparison between the original sequence length and the encoded floating-point value size which equals the double precision bit size. The system displays the compression ratio after completing the calculation. The system records the duration of arithmetic coding operations. The program generates Huffman codes and DNA sequence encodings and displays memory consumption and both compression efficiency metrics and complete processing duration.

IV. MEMORY ANALYSIS AND COMPRESSION EFFICIENCY

The evaluation of Huffman and Arithmetic coding implementations used synthetic DNA sequences for performance

assessment. The synthetic approach allowed us to remove biological pattern influence while testing the core compression algorithms and their data structures independently. The evaluation used two test sequences which included a 24-nucleotide short sequence (Ex 1) and a 156-nucleotide long sequence (Ex 2). The Huffman-based Binary Search Tree (BST) used nucleotide frequency to generate optimal prefix codes while Arithmetic coding used probability intervals for encoding. The evaluation assessed both methods through their memory usage and their ability to compress data. The Huffman coding system delivered substantial memory reductions through its efficient sequence compression process that required minimal computational effort. The processing time for Arithmetic coding exceeded Huffman coding but delivered comparable compression performance.

A. Memory Usage

The evaluation of compression efficiency requires a comparison between the storage needs of the original DNA sequence and its compressed version. The space reduction analysis through encoding techniques demonstrates the algorithm's ability to minimize storage needs.

Input Memory: Each character in the DNA sequence (8 bits for ASCII) contributes to the total memory requirement, as shown in Equation 1.

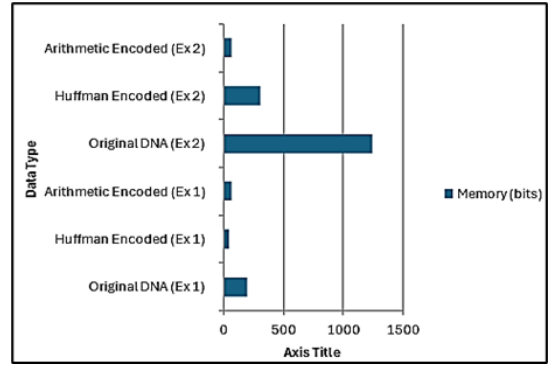
$$\text{Memory Input} = \text{Length of DNA sequence} \times 8 \text{ bits} \quad (1)$$

Output Memory: The length of the encoded string determines the memory used by the Huffman encoded sequence, as defined in Equation 2.

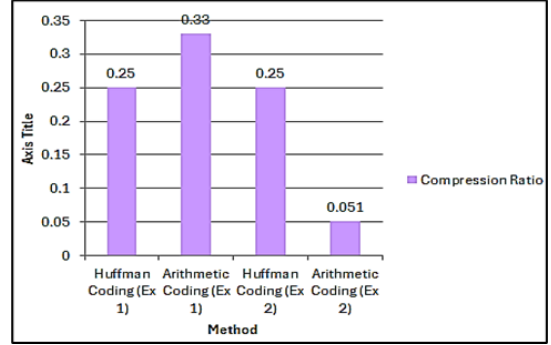
$$\text{Memory Output} = \text{Length of encoded DNA bits} \quad (2)$$

The analysis includes two specific cases to show actual memory reduction performance. The first example uses Huffman encoding to compress a 24-nucleotide DNA sequence through both methods. The memory needs for the original sequence amount to $24 \times 8 = 192$ bits. The Huffman encoding process reduced the binary sequence length to 48 bits which achieved a compression ratio of 0.25. The final encoded value requires 64 bits of storage as a double precision number which produces a compression ratio of 0.33.

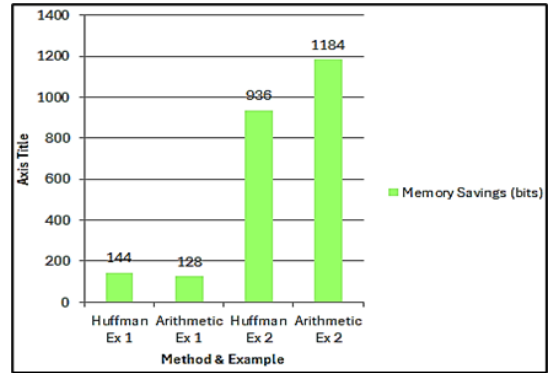
The analysis includes a 156-nucleotide DNA sequence as its second example. The sequence requires 1248 bits to store its original data. The Huffman encoding process generates a 312-bit binary output which achieves a 0.25 compression ratio. Huffman encoding produces a compressed binary output occupying 312 bits, again yielding a compression ratio of 0.25. Arithmetic encoding, however, still requires only 64 bits to represent the final encoded value, which corresponds to a much lower compression ratio of approximately 0.0512. This is a critical finding: Arithmetic Coding compresses the entire sequence into a single, fixed-length 64-bit double-precision floating-point number. For the longer sequence (Ex 2, 1248 bits original size), this constant memory footprint leads to a vastly superior compression ratio of approximately 0.051



(a) Original and Compressed Memory (bits)



(b) Compression Ratio (Output/Original Size)



(c) Memory Saved (bits)

Fig. 3: Graphical Representation of efficiency of Huffman vs. Arithmetic Encoding

($\approx 64/1248$). This low ratio significantly outperforms Huffman coding for longer sequences, confirming the theoretical advantage of Arithmetic Coding for large datasets where the fixed memory cost is highly amortized.

The memory usage comparison between the original sequence and the compressed outputs of both Huffman and Arithmetic coding appears in Figure 3a. The memory savings of both techniques become visible through the data presented in Figure 3c.

B. Compression Efficiency

The ability of data compression algorithms to minimize file size represents their compression efficiency.

TABLE I: Comparison of Memory Requirements and Compression Ratios for Huffman and Arithmetic Encoding of DNA Sequences

Ex.	Total Nucleotides	Memory Req. (bits)	Memory Req. Huffman (bits)	Huffman Comp. Ratio	Arithmetic Encoded Value	Memory Req. Arithmetic (bits)	Arithmetic Comp. Ratio
1	24	192	48	0.25	0.374722	64	0.33
2	156	1248	312	0.25	0.355425	64	0.051

The compression ratio equals the ratio between the data size after compression and its original size (see Equation 3).

$$\text{Compression Ratio} = \frac{\text{Compressed size (bits)}}{\text{Original size (bits)}} \quad (3)$$

The examples show that Huffman encoding works best for short DNA sequences because it produces compressed data that requires fewer than 64 bits. The method provides better performance for short sequences because it operates quickly and maintains simple operations. The method provides better performance for longer sequences because it produces fixed-length compressed data that stays within 64 bits. Despite requiring more computational resources, arithmetic encoding tends to produce better compression ratios for longer or more complex sequences. Therefore, the choice of encoding method should be guided by the sequence size and the desired trade-off between speed and compression efficiency.

Table I summarizes the memory requirements and compression ratios obtained from two DNA sequence examples. The compression results indicate that Huffman coding provides consistent ratios. The compression results indicate that Huffman coding provides consistent ratios (0.25), while Arithmetic coding shows variable results depending on the sequence⁷. The consistency of the 0.25 Huffman ratio is inherent to the balanced nature of the synthetic DNA sequences used, where the four nucleotides (A, C, G, T) appear with near-equal frequency. Under these conditions, Huffman's optimal prefix codes assign a fixed 2 bits per nucleotide (BPB) code to each symbol (since $\log_2(4) = 2$). Given the original ASCII representation is 8 bits per character, this results in a fixed theoretical compression ratio of $2/8 = 0.25$, while Arithmetic coding shows variable results depending on the sequence.

Figure 3b presents a graphical comparison of the compression ratios for both algorithms, further validating the tabular results in Table I.

V. PERFORMANCE AND ACCURACY EVALUATION

To ensure the reliability of the compression methods, we assessed the accuracy of DNA sequences after decompression. The Huffman and Arithmetic coding algorithms maintained lossless compression according to Figures 4 and 5 because they allowed perfect reconstruction of the original sequences without any information loss. The comparison between original and decompressed sequences through multiple tests proved that all nucleotide sequences matched perfectly. The maintenance of exact data fidelity stands as a vital requirement for bioinformatics because it enables reliable genomic analysis and interpretation.

```

Entered the DNA Sequence: AGTTCAAC
Huffman Encoded DNA Sequence: 1100101001111101

Decompression:

Huffman Decoding Steps:
Decoded 11 to A
Decoded 00 to G
Decoded 10 to T
Decoded 10 to T
Decoded 01 to C
Decoded 11 to A
Decoded 11 to A
Decoded 01 to C

Huffman Decoded DNA Sequence: AGTTCAAC

```

Fig. 4: Decoding Huffman DNA Sequence

```

Entered the DNA Sequence: AGTTCAAC

Decompression:

Arithmetic Decoding Steps:
Decoded value 0.279470 to A with interval [0.000000, 0.375000]
Decoded value 0.745255 to G with interval [0.625000, 0.750000]
Decoded value 0.962036 to T with interval [0.750000, 1.000000]
Decoded value 0.848145 to T with interval [0.750000, 1.000000]
Decoded value 0.392578 to C with interval [0.375000, 0.625000]
Decoded value 0.070312 to A with interval [0.000000, 0.375000]
Decoded value 0.187500 to A with interval [0.000000, 0.375000]
Decoded value 0.500000 to C with interval [0.375000, 0.625000]

Arithmetic Decoded DNA Sequence: AGTTCAAC

```

Fig. 5: Decoding Arithmetic DNA Sequence

VI. APPLICATION AND USE CASES

The two lossless data compression algorithms Huffman coding and Arithmetic coding function differently based on their operational efficiency and system complexity requirements.

The ZIP and GZIP compression tools use Huffman coding to shorten frequent symbols which results in smaller data sizes. The compression algorithm Huffman coding works with transformed data to compress multimedia formats through its application in JPEG image and MP3 audio compression. Huffman coding serves two purposes in communication systems by reducing data redundancy and improving error detection and correction capabilities.

The technique of Arithmetic coding outperforms Huffman coding because it encodes symbols as fractional bit lengths, achieving theoretical entropy limits. The modern multimedia codecs H.264, HEVC (H.265), and AV1 use Arithmetic coding for video and audio compression to achieve high-quality streaming at reduced bit rates. Arithmetic coding stands out in error-resilient systems through its tight data compression because it helps applications that need to correct small errors. JBIG2 uses Arithmetic coding for file compression to achieve better performance than Huffman coding under specific conditions. Both techniques function together in various practical

applications to achieve optimal performance through balanced compression efficiency and system complexity.

A. Limitations and Real-World Challenges

While both Huffman and Arithmetic coding realize very good lossless compression, there are certain limitations in practical applications. The first is scalability: currently, the provided implementations were tested on synthetic sequences of up to a moderate length of 156 nucleotides. Real-world genomic data can span billions of base pairs and would be difficult or impossible to handle by these techniques, instead requiring more complex, adaptive, and block-based schemes. Error resilience is a second significant differentiator. Because Arithmetic Coding reduces the entire sequence to a single fractional value, a single bit error in the encoded value during transmission would likely corrupt most or the entirety of the decoded sequence. Huffman coding gives much stronger localized error resilience, where one error would affect just one or two code words. The analysis of real-world genomic datasets from NCBI reveals complex non-random patterns which general-purpose symbol coding tools fail to detect at the level of domain-specific tools Biocompress and DNACompress.

VII. CONCLUSION

The paper conducts a comparative evaluation of Huffman and Arithmetic Coding methods for DNA sequence compression because bioinformatics requires better data management solutions. The implemented systems proved that both methods decrease genomic data storage requirements through lossless data preservation. The Binary Search Tree (BST)-based Huffman coding method delivered quicker encoding operations but Arithmetic coding produced better compression performance for datasets containing uneven nucleotide patterns.

These compression methods demonstrate their ability to boost genomic data storage and transmission performance which makes them essential tools for researchers who handle extensive genomic information. Researchers should develop new compression systems which unite the advantages of Huffman and Arithmetic coding to achieve better results in bioinformatics operations. The development of Huffman and Arithmetic Coding systems for real-time applications including streaming and IoT requires optimization. The integration of AI with these compression methods and their development for error-resistant networks and high-resolution media formats and quantum computing applications represents potential future research areas.

VIII. FUTURE WORK

The development of hybrid methods which use Huffman encoding for fast local pattern processing and Arithmetic Coding for better output compression will enhance results in various bioinformatics applications.

The development of optimized methods for real-time applications including streaming and IoT systems alongside integration with genomic compression software for specific domains represents potential future research directions.

ACKNOWLEDGMENT

The authors wish to extend their gratitude to Mrs. Minal Deshmukh for their guidance and support, throughout this research project. Their expertise and encouragement were instrumental in the completion of this study. We also want to acknowledge Vishwakarma Institute of Information Technology for providing the resources and infrastructure, for our research efforts.

REFERENCES

- [1] A. Gurunathan and F. B. K. Moideen, "Enhanced index based dna sequence compression algorithm," *International Journal of Intelligent Engineering & Systems*, vol. 17, no. 1, 2024.
- [2] A. Hassan, S. Javed, S. Hussain, R. Ahmad, and S. Qazi, "Arithmetic n-gram: An efficient data compression technique," 2024.
- [3] B. Kumari, N. K. Kamal, A. M. Sattar, and M. K. Ranjan, "Adaptive huffman algorithm for data compression using text clustering and multiple character modification," 2023.
- [4] A. A. Hussein, R. M. Al Baity, and S. A. Hadi, "Randomized information hiding in rgb images using genetic algorithm and huffman coding," *Revue d'Intelligence Artificielle*, vol. 37, no. 6, pp. 1435–1440, 2023.
- [5] M. Alost and A. Souri, "Design of effective lossless data compression technique for multiple genomic dna sequences," 2022.
- [6] M. H. Sarkar and A. F. Ashrafi, "Genetic sequence compression using machine learning and arithmetic encoding decoding technique," 2022.
- [7] D. Gomathi, I. Rajathi, and C. Gopala Krishnan, "Dna compression using rle and huffman encoder," 2021.
- [8] M. Ganapathy, "It reduction-based compression algorithm for dna sequences," 2021.
- [9] J. Zhao, Z. Ma, and F. Xie, "An improved huffman-based compression algorithm for large-scale dna sequences," 2021.
- [10] P. Mishra, C. Bhaya, A. K. Pal, and A. K. Singh, "Compressed dna coding using minimum variance huffman tree," 2021.
- [11] B. Pulova-Mihaylova, I. Mihaylov, I. Avdjieva, and D. Vassilev, "A system for compression of sequencing data," *International Journal of Intelligent Engineering & Systems*, vol. 17, no. 1, 2024.
- [12] A. Jahaan, T. N. Ravi, and S. P. Arokiaaraj, "Comparative study and survey on existing dna compression techniques," 2017.
- [13] V. Machhi and M. S. Patel, "Compression techniques applied to dna data of various species," *International Journal of Bio-Science and Bio-Technology*, vol. 8, no. 3, pp. 45–52, 2016.
- [14] U. Ghoshdastider and B. Saha, "Genomecompress: A novel algorithm for dna compression," 2005.
- [15] B. Behzadi and F. Le Fessant, "Dna compression challenge revisited: A dynamic programming approach," in *Annual Symposium on Combinatorial Pattern Matching*. Springer Berlin Heidelberg, 2005, pp. 190–200.
- [16] D. Xu, M. Qiu, and T. Yi, "A comparative study on the performance of huffman coding and arithmetic coding," *Academic Journal of Engineering and Technology Science*, vol. 8, no. 3, pp. 1–7, 2025.
- [17] G. Xu, Z. Fan, Y. Li, and H. Sun, "Toward a better compression for dna sequences using huffman encoding," *PMC*, vol. 53, no. 7, pp. 147–156, 2017.
- [18] S. Elnady, S. Sayed, and A. Salah, "Hadc: A hybrid compression approach for dna sequences," *IEEE Access*, vol. 10, pp. 104 723–104 732, 2022.
- [19] A. Diss and D. Pradas, "Lossless dna compression," Ph.D. dissertation, University of Cambridge, 2019.